



HealBot

김재훈, 남윤우, 신선호, 임동균

2025.12.09

* 목차

1. 개발 배경

2. 팀 소개

3. 구현 목표

4. 요구사항 정의서

5. ERD

6. 개발 환경

7. 메인페이지

8. 코드 리뷰

9. 시연

10. 소감

* 개발 배경

건강·의학 > 건강일반

은평성모병원, “이런 증상은 무슨 과를 가야 할까요?” 환자 물음에 답하다

매주 금요일 증상별로 진료하는 ‘금요일 증상 클리닉 데이’ 운영 시작

임혜정 기자 | 기사입력 : 2021-03-17 09:49

가 가 금

가톨릭대학교 은평성모병원(병원장 권순용)은 환자들이 병원을 찾기 전 가장 고민하는 “이런 증상은 무슨 과를 가야 할까요?”에 대한 해답을 제시하기 위해 ‘금요일 증상 클리닉 데이’를 본격 시행한다고 밝혔다.

가톨릭대학교은평성모병원,금요일증상클리닉데이

은평성모병원이 3월부터 매주 금요일 시행하는 ‘금요일 증상 클리닉 데이’는 증상은 있지만 무슨 진료과를 찾아가야할지 몰라 증상을 방치하거나 병원을 늦게 찾는 환자들이 많다는 점을 고려해 대표 증상 39개를 선정하고, 증상별로 해당 진료과를 바로 연결해 환자들이 빠르고 정확한 진료를 받을 수 있도록 돕는다는 설명이다.

단순히 증상별로 진료과를 연결하는 것을 넘어 환자들의 진료 만족도를 높이기 위해 증상을 세분화해 맞춤형 진료를 시행하는 것도 ‘금요일 증상 클리닉 데이’의 특징이다.

예를 들어 다양한 근골격계 통증에 대해 관절통증은 류마티스내과, 목·허리통증은 재활의학과, 다리통증과 부종은 혈관이식외과, 손목·발목통증과 방사통(팔다리, 허리 쪽으로 뻗어 나가는 통증)은 정형외과, 만성통증은 마취통증의학과에서 우선 진료하고 필요한 경우 협진을 통해 치료계획을 수립하는 식이다.

은평성모병원 조현희 외래부장(산부인과 교수)은 “환자들이 불편함을 느끼는 증상은 성별, 연령별, 기저질환별, 생활습관별로 다양하게 발생하기 때문에 증상의 원인을 찾는 과정에서부터 어려움을 겪을 수 있다”면서 “증상 클리닉 운영을 통해 환자들이 쉽게 병원을 찾고 빠르게 치료를 받을 수 있길 기대한다”고 말했다.

5. 이럴 땐 어느 과에서 무슨 진료를 받아야 할까?

한 가지 증상을 두고도 다양한 원인이 있을 수 있으므로 어느 진료과를 선택해야 한다고 단정 짓기는 어렵다 .

이럴 때는 국민건강보험에서 운영하는 ‘건강 in’ 홈페이지 (<http://hi.nhis.or.kr>) 에 방문해보자 .

‘건강 in’ 에 접속하면 먼저 일반적으로 자주 나타나는 일반증상과 여성이나 남성 , 어린이에게 나타나는 주요 증상들을 보고

자기 상태에 해당되는 항목을 클릭하며 질문에 답을 하다보면 선택에 따라 의심되는 병명들이 나온다 .

또 ‘바로가기’ 버튼을 누르면 해당 질병이나 증상 , 치료법에 대해 자세히 볼 수 있다 .

일반인이 질병 증상을 느낄 때, 어느 진료과를 방문해야 할지 판단하기 어렵다.

잘못된 진료과 방문은 시간 낭비와 추가 비용을 발생시킨다.

이에 사용자가 입력한 증상을 기반으로 예상 질환과 적절한 진료과를 추천하여, 효율적인 병원 방문을 돕는 시스템을 개발하고자 한다.

* 개발 배경

PROCEEDINGS OF HCI KOREA 2024 학술대회 발표 논문집

The discrepancies between the diagnostic range of the medical specialty and the symptoms of patients may be associated with enormous costs for both the patient and the hospital. This work proposes a medical specialty prediction model based on domain-specific pre-trained BERT from health question texts. The proposed model showed higher predictive performance in the experiments compared with four competitive deep-learning-based NLP models. We expect that the proposed model can relieve the discrepancy problem by suggesting suitable specialties from the text and can be utilized by hospitals or healthcare institutions (Kim et al., 2023, p. 6).

YOOJOONG KIM, JONG-HO KIM, YOUNG-MIN KIM, SANGHOUN SONG, HYUNG JOON JOO. (2023). MEDICAL SPECIALTY PREDICTION MODEL BASED ON DOMAIN-SPECIFIC PRE-TRAINED BERT FROM HEALTH QUESTION TEXTS. INTERNATIONAL JOURNAL OF MEDICAL INFORMATICS, 170, 104956.

의료 진단 범위와 환자 증상 간의 불일치는 환자와
병원 모두에게 막대한 비용 부담으로 이어 질 수 있음 따라서 AI기반 의료 전문 분야 예측 모델을 통해 불일치 문제를 해소한 논문

* 팀 소개



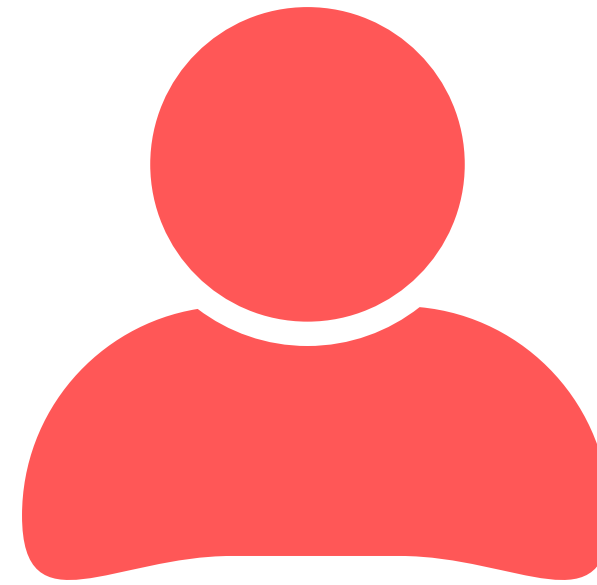
김재훈

관리자 페이지, 대시
보드, 회원관리, 병
원관리, 질환관리,
공지사항, 신고관리,
커뮤니티, 리뷰관리



남윤우

섹션기반 메인페이지
건강정보 모달시스템
카카오 API 기반 병원 표시
지능형 병원 검색 시스템
병원 상세 정보 모달



신선호

마이페이지, 프로필
정보, 내가 작성한
글, 재재 내역, 회원
탈퇴, 커뮤니티 페이
지, 리뷰 페이지, 신고
하기



임동균

소셜 로그인 기능
시활용 증상 검색기능
병원 및 질환 데이터
크롤링 및 정규화
증상 기반 질환 매핑
알고리즘 구현

* 구현 목표

본 프로젝트의 목표는 사용자가 느끼는 증상을 기반으로 적절한 진료과를 안내하고, 불필요한 병원 방문을 최소화하여 시간과 비용을 절감할 수 있도록 돕는 것이다. 또한, AI 기반 자연어 처리와 위치 기반 병원 검색, 커뮤니티와 건강정보 콘텐츠, 영수증 OCR 인증 리뷰 시스템을 통합하여 사용자가 신뢰할 수 있는 병원 정보와 실제 방문 경험을 제공하는 효율적인 의료 서비스 지원 플랫폼을 구축하는 것을 최종 목표로 한다.

1. 증상 기반 질환 예측 - 사용자가 선택한 증상을 분석하여 예상 질환 제시 - 질환별 적합한 진료과 추천
2. AI 자연어 처리 - GEMINI API를 활용한 증상 자동 추출 - 예: "머리가 아프고 열나" → "두통", "발열"로 변환
3. 위치 기반 병원 추천 - 사용자 위치 기반 인근 병원 검색 - 카카오맵 API 연동
4. 리뷰 시스템 - 실제 방문 인증(영수증) 기반 신뢰도 높은 리뷰 - OCR API기반 인증

* 구현 목표 - 차별점

기존사이트

- 닥터나우: 비대면 진료 중심.
증상 입력은 있지만 진료과 추천 기능 없음

차별점

- 자연어 기반 증상 서술(NLP)을 분석하여 가장 증상을 DB와 매칭
매칭된 증상을 기반으로 질환 예측



비대면 진료

병원 찾기

비급여 가격

건강 정보

실시간 의료 상담

앱 다운로드

소아청소년과	이비인후과	탈모 진료비	건강 매거진	실시간 질문
감기/독감	피부과	비만 진료비	1분 FAQ	질문하기
인공눈물	내과	여드름 진료비	의약품 사전	
탈모	산부인과	최저가 약국	질환백과	
여드름	가정의학과			
전체 보기	전체 보기			



비대면 진료 시작하기



감기/독감



소아과



내과



산부인과



긴장완화



탈모



여드름



다이어트



인공눈물



전체 보기

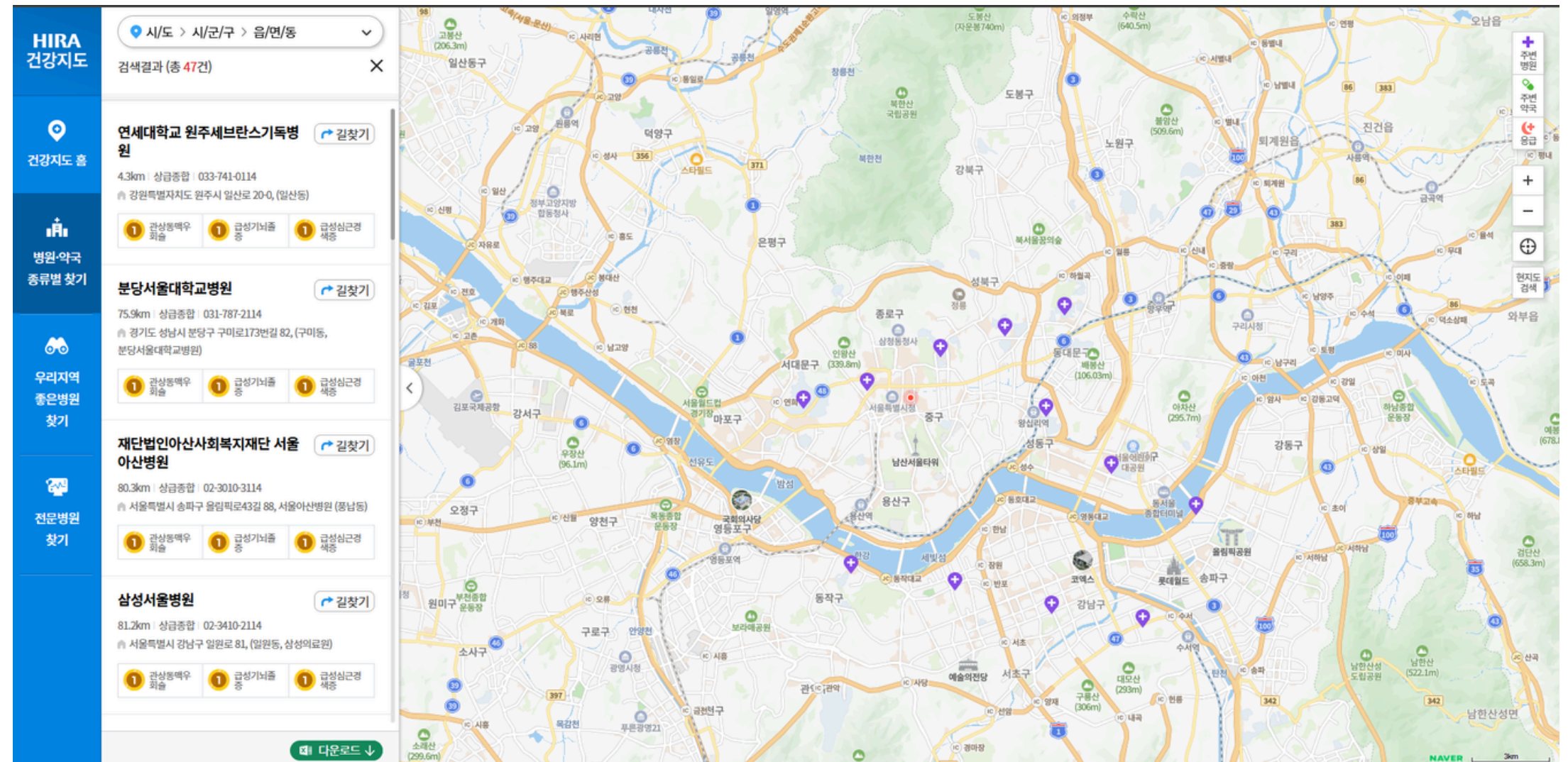
* 구현 목표 - 차별점

기존사이트

- 건강보험심사평가원 병원 찾기:
진료과 기반 병원찾기 기능 제공

차별점

- 증상 기반으로 진료과를 추천하여
병원 찾기 기능 제공



* 요구사항 정의서

증상 입력 및 분석 기능

- 신체 부위별 분류(머리, 눈, 코, 목, 가슴, 배 등)
- 증상 유형별 분류(두통, 발열, 팔 저림, 다리 저림, 반점 등)
- 복수의 증상을 최소 1개 ~ 최대 10개까지 동시에 선택 가능

AI 자연어 처리 기능

- GEMINI API를 통한 텍스트 분석(예 : “머리가 깨질듯이 아파요” → ‘두통’)
- 자연어 입력 시 관련 증상 키워드를 자동으로 추출

질환 예측 및 진료과 추천 기능

- 증상 입력 및 AI 자연어 처리 기능을 통한 증상 키워드 추출로 나온 결과를 기반으로 예상 질환 목록 제시
- 예상 질환 목록별 적합한 진료과를 추천

* ERD

병원진료과ID	DEPT_ID	Domain	NUMBER	NOT NULL	Default value	Comment
병원ID	HOSPITAL_ID	Domain	VARCHAR2(20)	NOT NULL	Default value	Comment
진료과	DEPARTMENT	Domain	VARCHAR2(50)	NULL	Default value	Comment

로그ID	LOG_ID	Domain	Type	NOT NULL	Default value	Comment
회원ID	MEMBER_ID	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
접속일	ACCESS_DATE	Domain	DATE	NULL	SYSDATE	Comment
클라이언트 주소	IP_ADDRESS	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
브라우저 정보	USER_AGENT	Domain	VARCHAR2(500)	NOT NULL	Default value	Comment

댓글ID	COMMENT_ID	Domain	NUMBER	NOT NULL	Default value	Comment
게시글ID	POST_ID	Domain	NUMBER	NOT NULL	Default value	Comment
회원ID	MEMBER_ID	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
댓글 내용	CONTENT	Domain	VARCHAR2(1000)	NOT NULL	Default value	Comment
등록일	CREATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment
수정일	UPDATE_AT	Domain	DATE	NULL	Default value	Comment
커뮤니티 상태	STATUS	Domain	VARCHAR2(20)	NOT NULL	'ACTIVE'	Comment

병원ID	HOSPITAL_ID	Domain	VARCHAR2(20)	NOT NULL	Default value	Comment
병원 이름	HOSPITAL_NAME	Domain	VARCHAR2(100)	NULL	Default value	Comment
병원 주소	ADDRESS	Domain	VARCHAR2(500)	NULL	Default value	Comment
병원 등급	HOSPITAL_GRADE	Domain	NUMBER	NULL	Default value	1 = 1차병원, 2 = 2차병원, 3 = 3차병원
병원 구분	HOSPITAL_TYPE	Domain	VARCHAR2(20)	NULL	Default value	(병원, 종합병원, 현병병원, 한의원, 치과)
병원 상세 설명	DETAILS	Domain	VARCHAR2(255)	NULL	Default value	병원에서 등록하지 않았습니다.
운영 시간	OPERATING_HOURS	Domain	VARCHAR2(255)	NULL	Default value	Comment
점심시간	LUNCH_TIME	Domain	VARCHAR2(20)	NULL	Default value	Comment
응급실 유무	EMERGENCY_YN	Domain	VARCHAR2(1)	NULL	'N'	Comment
대표 전화번호	PHONE	Domain	VARCHAR2(20)	NULL	Default value	Comment
응급실 전화번호	ER_PHONE	Domain	VARCHAR2(20)	NULL	Default value	Comment
경도	LONGITUDE	Domain	NUMBER(11,8)	NULL	Default value	Comment
위도	LATITUDE	Domain	NUMBER(10,8)	NULL	Default value	Comment
간이지도	SIMPLE_MAP	Domain	VARCHAR2(255)	NULL	Default value	Comment

회원ID	MEMBER_ID	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
로그인 타입	LOGIN_TYPE	Domain	VARCHAR2(20)	NOT NULL	Default value	일반/카카오/네이버
소셜ID	SOCIAL_ID	Domain	VARCHAR2(100)	NULL	Default value	Comment
비밀번호	PASSWORD	Domain	VARCHAR2(255)	NULL	Default value	Comment
회원명	USER_NAME	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
회원이메일	EMAIL	Domain	VARCHAR2(100)	NOT NULL	Default value	Comment
회원 전화번호	PHONE	Domain	VARCHAR2(20)	NOT NULL	Default value	Comment
회원 생년월일	BORN_DATE	Domain	NUMBER	NULL	Default value	Comment
회원 주소	ADDRESS	Domain	VARCHAR2(500)	NULL	Default value	Comment
회원 성별	GENGER	Domain	VARCHAR2(1)	NULL	Default value	Comment
회원 가입일	CREATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment
관리자 권한 유무	ADMIN_YN	Domain	VARCHAR2(1)	NOT NULL	'N'	Comment

신고ID	REPORT_ID	Domain	NUMBER	NOT NULL	Default value	Comment
글 종류	TARGET_TYPE	Domain	VARCHAR2(20)	NOT NULL	Default value	Comment
게시글ID	POST_ID	Domain	NUMBER	NOT NULL	Default value	Comment
댓글ID	COMMENT_ID	Domain	NUMBER	NOT NULL	Default value	Comment
신고자ID	REPORTER_ID	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
신고 종류	REASON_TYPE	Domain	VARCHAR2(30)	NOT NULL	Default value	Comment
신고 상세 내용	DETAIL	Domain	VARCHAR2(1000)	NULL	Default value	Comment
신고 상태	STATUS	Domain	VARCHAR2(20)	NOT NULL	'PENDING'	Comment
등록일	CREATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment
관리자 답변	REPLY	Domain	VARCHAR2(1000)	NULL	Default value	Comment

증상ID	SYMPTOM_ID	Domain	NUMBER	NOT NULL	Default value	Comment
증상명	SYMPTOM_NAME	Domain	VARCHAR2(100)	NULL	Default value	Comment
증상 정보	SYMPTOM_DETAIL	Domain	VARCHAR2(4000)	NULL	Default value	Comment

질환증상ID	MAPPING_ID	Domain	NUMBER	NOT NULL	Default value	Comment
질환명	DISEASE_NAME	Domain	VARCHAR2(200)	NOT NULL	Default value	Comment
증상명	SYMPTOM_NAME	Domain	VARCHAR2(100)	NOT NULL	Default value	Comment

질환ID	DISEASE_ID	Domain	NUMBER	NOT NULL	Default value	Comment
질환명	DISEASE_NAME	Domain	VARCHAR2(200)	NULL	Default value	Comment
이미지URL	IMAGE_URL	Domain	VARCHAR2(255)	NULL	Default value	Comment
설명	DESCRIPTION	Domain	VARCHAR2(4000)	NULL	Default value	Comment
총 환자 수	TOTAL_PATIENTS	Domain	NUMBER	NULL	Default value	Comment

리뷰ID	REVIEW_ID	Domain	NUMBER	NOT NULL	Default value	Comment
병원ID	HOSPITAL_ID	Domain	VARCHAR2(20)	NOT NULL	Default value	Comment
회원ID	MEMBER_ID	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
별점	SCORE	Domain	NUMBER	NOT NULL	Default value	Comment
리뷰 내용	CONTENT	Domain	VARCHAR2(2000)	NOT NULL	Default value	Comment
작성일	CREATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment

공지사항 번호	NOTICE_NO	Domain	NUMBER	NOT NULL	Default value	Comment
공지사항 제목	NOTICE_SUBJECT	Domain	VARCHAR2(300)	NULL	Default value	Comment
공지사항 내용	NOTICE_CONTENT	Domain	VARCHAR2(4000)	NULL	Default value	Comment
공지사항 작성자	NOTICE_WRITER	Domain	VARCHAR2(30)	NULL	Default value	Comment
공지사항 작성일	WRITE_DATE	Domain	TIMESTAMP	NULL	SYSTIMESTAMP	Comment
공지사항 조회수	VIEW_COUNT	Domain	NUMBER	NULL	0	Comment
카테고리	CATEGORY	Domain	VARCHAR2(20)	NULL	Default value	Comment
등록일	CREATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment
수정일	UPDATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment

게시글ID	POST_ID	Domain	NUMBER	NOT NULL	Default value	Comment
회원ID	MEMBER_ID	Domain	VARCHAR2(50)	NOT NULL	Default value	Comment
카테고리	CATEGORY	Domain	VARCHAR2(20)	NOT NULL	Default value	Comment
커뮤니티 제목	TITLE	Domain	VARCHAR2(200)	NOT NULL	Default value	Comment
커뮤니티 내용	CONTENT	Domain	CLOB	NOT NULL	Default value	Comment
커뮤니티 조회수	VIEWS	Domain	NUMBER	NOT NULL	0	Comment
등록일	CREATED_AT	Domain	DATE	NOT NULL	SYSDATE	Comment
수정일	UPDATED_AT	Domain	DATE	NULL	Default value	Comment
커뮤니티 상태	STATUS	Domain	VARCHAR2(20)	NOT NULL	'ACTIVE'	Comment

병원, 증상
회원, 커뮤니티, 리뷰
총 13개의 테이블

특정별 질환 번호	FEATURED_DISEASES_NO	Domain	NUMBER	NOT NULL	Default value	Comment
질환명	DISEASE_NAME	Domain	VARCHAR2(200)	NOT NULL	Default value	Comment
표시 순서	DISPLAY_ORDER	Domain	NUMBER	NOT NULL	Default value	Comment
작성일	CREATED_AT	Domain	TIMESTAMP	NULL	CURRENT_TIMESTAMP	Comment

* 개발환경

프론트엔드



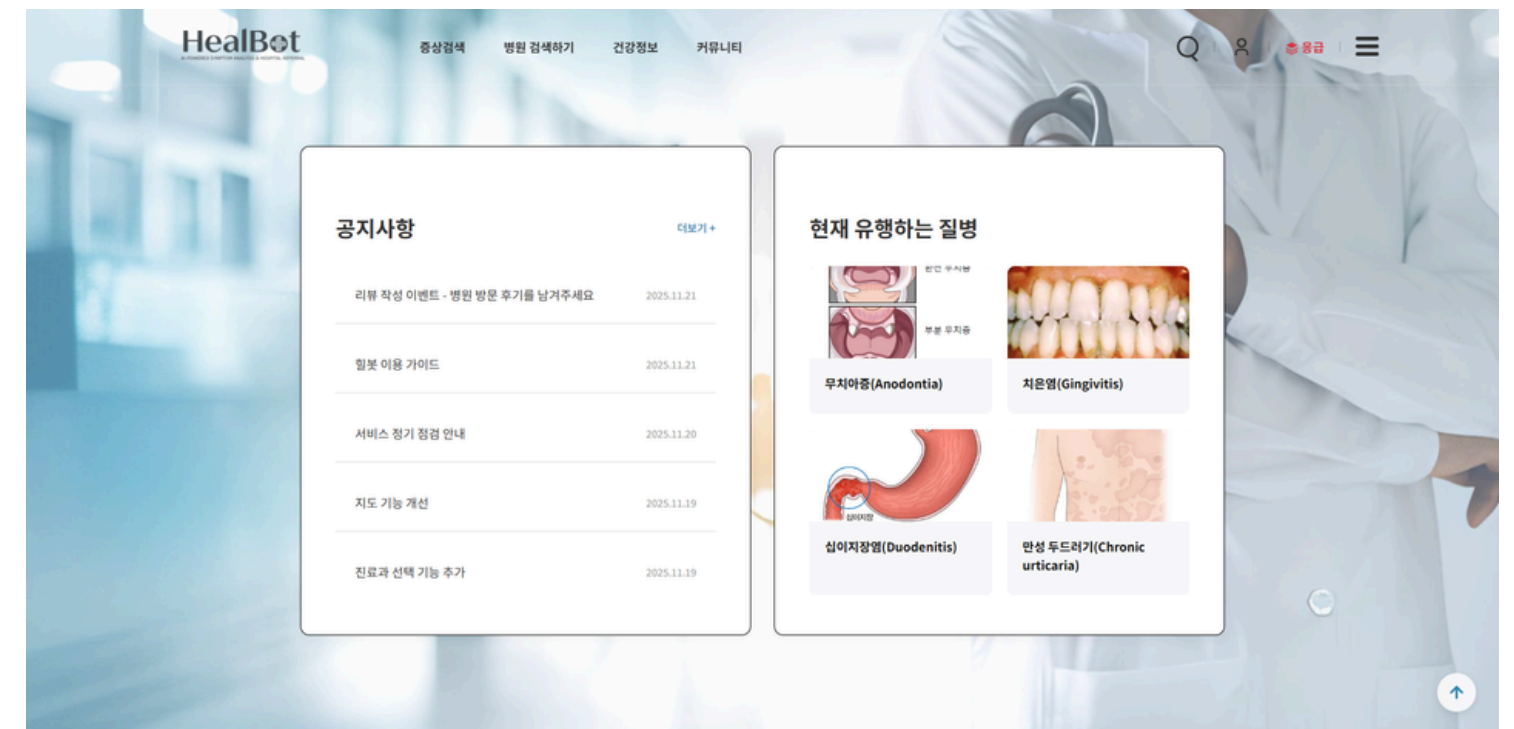
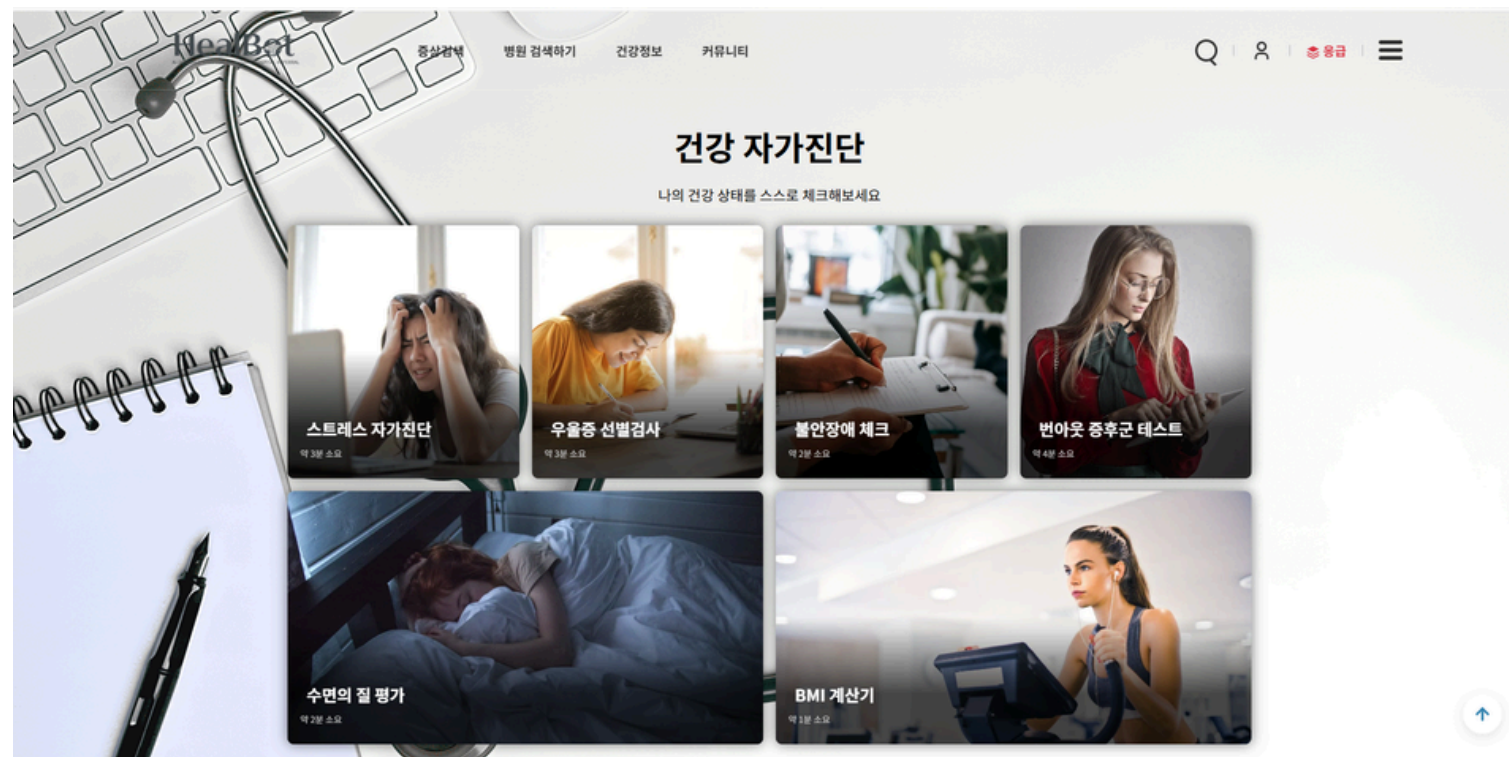
백엔드



DB / VCS



* 메인페이지



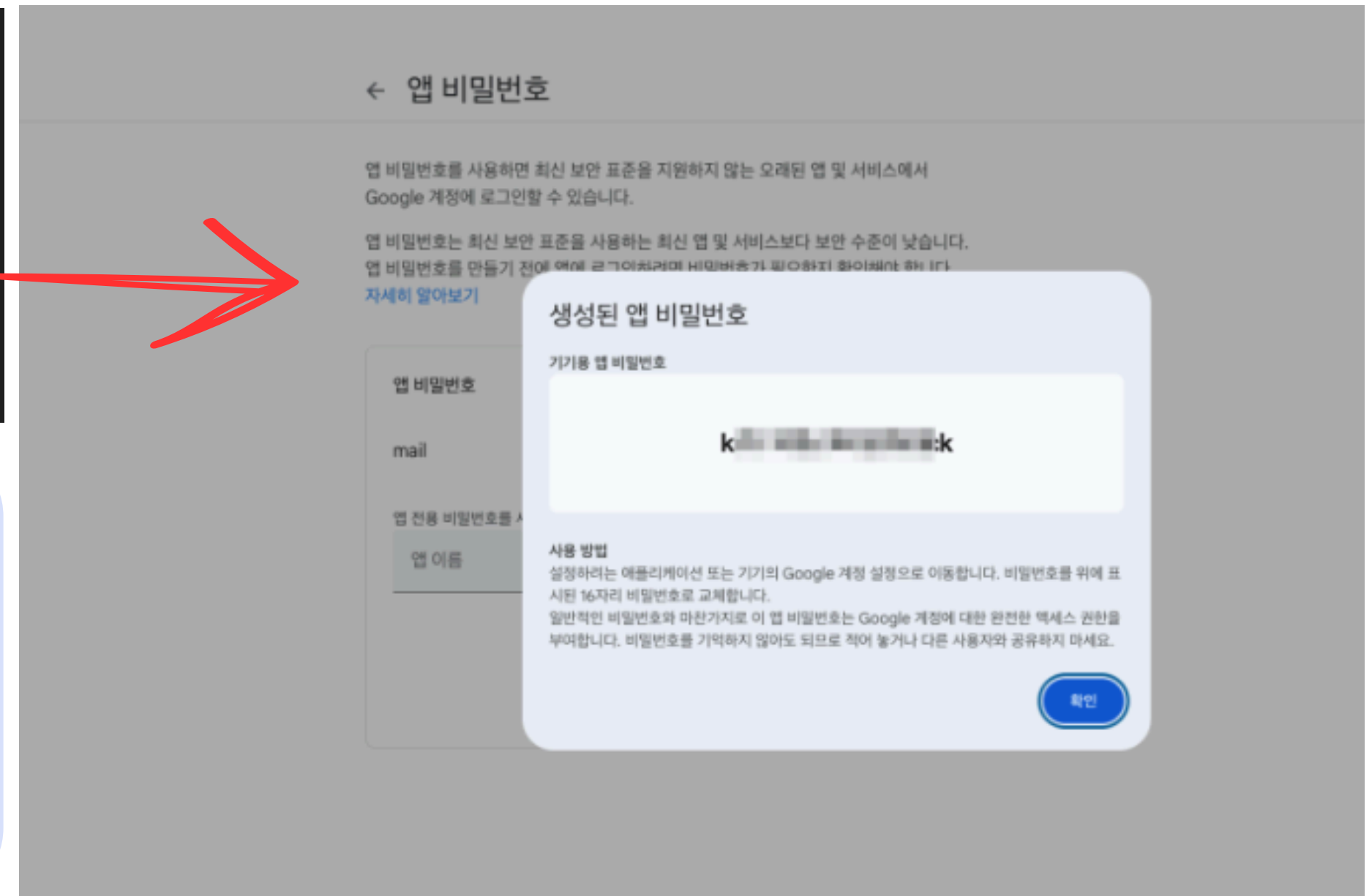
* 코드리뷰 - 김재훈

이메일 인증 - SMTP

application.properties 설정

```
# Gmail SMTP Configuration
1 spring.mail.host=smtp.gmail.com
2 spring.mail.port=587
3 spring.mail.username=healbot.official@gmail.com
4 spring.mail.password=
5 spring.mail.properties.mail.smtp.auth=true
6 spring.mail.properties.mail.smtp.starttls.enable=true
```

- 1 host : 메일을 전송할 SMTP 서버의 호스트 이름
- 2 port : SMTP 서버가 수신하는 포트 번호
- 3 username : SMTP 인증에 사용할 사용자 이름 (실제 발신자 계정 이메일 주소)
- 4 password : 발신자 계정의 앱 비밀번호 (2단계 인증을 사용한 앱 비밀번호)
- 5 auth : SMTP서버가 인증(로그인)을 요구하는 여부
- 6 starttls.enable : STARTLS를 활성화 여부
(TRUE이면 포트 587에서 암호화된 연결로 전환하여 전송 보안을 확보)



* 코드리뷰 - 김재훈 이메일 인증 - SMTP

MemberController (이메일 발송)

```
/**
 * 이메일 인증코드 발송
 */
1 @PostMapping("/send-email-code")
public ResponseEntity<?> sendEmailCode(@RequestParam String email) {
2     try {
        boolean sent = emailService.sendVerificationCode(email);

3         return ResponseEntity.ok(new java.util.HashMap<String, Boolean>() {{
            put("success", sent);
        }});
    } catch (Exception e) {
        e.printStackTrace();
4         return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new java.util.HashMap<String, Boolean>() {{
                put("success", false);
            }});
    }
}
```

- 1 POST /send-email-code 요청을 처리하는 메서드
요청 파라미터로 email을 받음
- 2 emailService.sendVerificationCode() 호출
실제 이메일 발송 로직을 담당하는 서비스 메서드
이메일 성공 시 true, 실패 시 false 반환
- 3 HTTP 200(OK)로 JSON 응답을 반환
- 4 서버 오류 발생 시 500 에러를 반환

* 코드리뷰 - 김재훈

이메일 인증 - SMTP

EmailService (이메일 발송)

```
/**
 * 인증코드 생성 및 이메일 발송
 */
public boolean sendVerificationCode(String email) {
    try {
        // 6자리 랜덤 인증코드 생성
        ① String code = generateRandomCode();

        // 이메일 메시지 작성
        ② SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom(fromEmail);
        message.setTo(email);
        message.setSubject("[HealBot] 이메일 인증 코드");
        message.setText(
            "안녕하세요, HealBot 입니다.\n\n" +
            "회원가입을 위한 인증 코드는 다음과 같습니다:\n\n" +
            code + "\n\n" +
            "이 코드는 5분간 유효합니다.\n\n" +
            "본인이 요청하지 않은 경우 이 메일을 무시하시기 바랍니다."
        );
    }
}
```

① 인증에 사용할 6자리 숫자 코드 생성

② 실제 메일을 전송
Gmail SMTP 설정에 의해 전송

* 코드리뷰 - 김재훈 이메일 인증 - SMTP

EmailService (이메일 발송)

```
// 이메일 발송
③ mailSender.send(message);

// 인증코드 저장 (5분 = 300000ms)
④ long expiryTime = System.currentTimeMillis() + 300000;
  verificationCodes.put(email, new VerificationCode(code, expiryTime));

  return true;
} catch (Exception e) {
  e.printStackTrace();
  return false;
}
}
```

③ 이메일 제목, 내용, 발신자, 수신자를 설정
이메일 본문에는 인증코드를 포함하여 설정

④ 인증코드를 메모리에 저장
5분 후 만료되도록 시간을 저장

* 코드리뷰 - 김재훈 이메일 인증 - SMTP

MemberController (이메일 검증)

```
/**
 * 이메일 인증코드 검증
 */
1 @PostMapping("/verify-email-code")
public ResponseEntity<?> verifyEmailCode(
    @RequestParam String email,
    @RequestParam String code) {
2 try {
    boolean verified = emailService.verifyCode(email, code);

3 return ResponseEntity.ok(new java.util.HashMap<String, Boolean>() {{
    put("verified", verified);
    }});
    } catch (Exception e) {
        e.printStackTrace();
4 return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(new java.util.HashMap<String, Boolean>() {{
        put("verified", false);
        }});
    }
}
```

- 1 POST /verify-email-code 요청을 처리하는 메서드
요청 파라미터로 email, code를 받아옴
사용자가 입력한 인증 코드를 받아서 검증
- 2 emailService.verifyCode() 호출
저장된 인증코드와 사용자가 입력한 코드를 비교
맞으면 true, 틀리면 false를 반환
- 3 HTTP 200(OK)로 JSON 응답을 반환
- 4 서버 오류 발생 시 500 에러를 반환

* 코드리뷰 - 김재훈

이메일 인증 - SMTP

EmailService (이메일 검증)

```
/**
 * 인증코드 검증
 */
1 public boolean verifyCode(String email, String code) {
    VerificationCode storedCode = verificationCodes.get(email);

    2 if (storedCode == null) {
        return false;
    }

    // 만료 시간 체크
    3 if (System.currentTimeMillis() > storedCode.getExpiryTime()) {
        verificationCodes.remove(email);
        return false;
    }

    // 코드 일치 여부 체크
    4 if (storedCode.getCode().equals(code)) {
        // 인증 성공 시 저장된 코드 삭제
        verificationCodes.remove(email);
        return true;
    }

    return false;
}
```

① 저장된 이메일 인증코드를 가져옴
없으면 인증 요청이 없는 상태임으로 code는 false 상태

② 인증 코드가 없을 때 false로 반환

③ 현재 시간이 만료시간을 초과했다면
→ 인증코드를 삭제하고 false로 반환

④ 인증코드가 사용자가 입력한 코드와 같으면 true를 반환
일회성 인증임으로 성공하면 저장된 인증코드를 삭제

* 코드리뷰 - 김재훈

접속 로그

MemberController (접속 로그 저장)

```
if (member != null) {  
    // 로그인 성공 시 세션에 memberId 저장  
    session.setAttribute("memberId", member.getMemberId());  
    response.setSuccess(1);  
  
    // 회원 접속 로그 저장 (관리자가 아니면)  
    ① if (!"Y".equals(member.getAdminYn())) {  
        saveAccessLog(member.getMemberId(), request);  
    }
```

```
/**  
 * 접속 로그 저장 헬퍼 메서드  
 */  
2 private void saveAccessLog(String memberId, HttpServletRequest request) {  
    3 try {  
        AccessLog accessLog = new AccessLog();  
        accessLog.setMemberId(memberId);  
        accessLog.setIpAddress(getClientIp(request));  
        accessLog.setUserAgent(request.getHeader("User-Agent"));  
        4 alService.saveAccessLog(accessLog);  
    } catch (Exception e) {  
        // 로그 저장 실패해도 로그인 프로세스는 정상 진행  
        e.printStackTrace();  
    }  
}
```

① 로그인 성공 시 일반 사용자의 접속 기록 저장

② DB ACCESS_LOG 테이블에 들어갈 AccessLog 객체를 생성

③ memberId, ipAddress, userAgent 등 접속 정보를 입력

④ AccessLogService 호출

* 코드리뷰 - 김재훈

접속 로그

AccessLogScheduler (7일 기준 로그 자동 삭제)

```
1 @Slf4j
  @Component
  @RequiredArgsConstructor
  public class AccessLogScheduler {

2     private final AccessLogService accessLogService;

3     @PostConstruct
    public void onStartup() {
        log.info("애플리케이션 시작 시 오래된 접속 로그 삭제");
        deleteOldLogs();
    }

4     @Scheduled(cron = "0 0 0 * * *")
    public void scheduledDelete() {
        log.info("오래된 접속 로그 삭제 스케줄러 실행");
        deleteOldLogs();
    }

5     private void deleteOldLogs() {
        int result = accessLogService.deleteOldAccessLogs();
        log.info("총 {}개의 오래된 접속 로그가 삭제되었습니다.", result);
    }
}
```

1 @Slf4j : log.info()같은 로그 출력을 위한 Lombok 어노테이션
@Component : 스케줄러가 자동 실행되려면 Spring IoC 컨테이너에 등록
@RequiredArgsConstructor : final 필드를 생성자 주입하도록 자동 생성

2 데이터베이스의 오래된 접속 로그를 삭제하는 기능을 호출하기 위한 필드

3 @PostConstruct : 서버 시작 시 딱 한 번 실행하는 어노테이션

4 @Scheduled : 매일 자정마다 실행하는 어노테이션

5 AccessLogService를 통해 오래된 접속 로그를 삭제

* 코드리뷰 - 김재훈

접속 로그

AdminController (일별 로그인 횟수 조회)

```
/**
 * 일별 로그인 횟수 조회 (최근 7일)
 * @return 일별 로그인 횟수
 */
① @GetMapping("/stats/daily-login")
② public List<Map<String, Object>> getDailyLoginCount() {
    return alService.getDailyLoginCount();
}
```

① GET /api/admin/stats/daily-login을 호출

② 날짜마다 로그인 횟수를 담아야 해서 Map 형태를 사용

* 코드리뷰 - 김재훈

접속 로그

accesslog-mapper.xml (각 호출되는 매서드에 대한 쿼리문)

```
<!-- 회원 접속 로그 저장 -->
<insert id="insertAccessLog" parameterType="com.hospital.boot.domain.accesslog.model.vo.AccessLog">
  <selectKey keyProperty="logId" resultType="int" order="BEFORE">
    SELECT SEQ_ACCESS_LOG_ID.NEXTVAL FROM DUAL
  </selectKey>
  INSERT INTO ACCESS_LOG (LOG_ID, MEMBER_ID, ACCESS_DATE, IP_ADDRESS, USER_AGENT)
  VALUES ({logId}, {memberId}, SYSDATE, {ipAddress}, {userAgent})
</insert>
```

① SEQ_ACCESS_LOG_ID.NEXTVAL

→ Oracle 시퀀스로 새로운 로그 번호를 미리 생성

INSERT INTO ACCESS_LOG : ACCESS_LOG 테이블에 값을 저장

→ 회원 접속 로그를 저장하는 INSERT문

* 코드리뷰 - 김재훈

접속 로그

accesslog-mapper.xml (각 호출되는 매서드에 대한 쿼리문)

2

```
<!-- 관리자 일별 로그인 횟수 조회 (최근 7일 기준) -->
<select id="getDailyLoginCount" resultType="map">
    SELECT
        TO_CHAR(AL.ACCESS_DATE, 'MM/DD') AS "date",
        COUNT(*) AS "count"

    FROM ACCESS_LOG AL

    INNER JOIN MEMBER M ON AL.MEMBER_ID = M.MEMBER_ID

    WHERE AL.ACCESS_DATE >= TRUNC(SYSDATE) - 7
        AND NVL(M.ADMIN_YN, 'N') = 'N'

    GROUP BY TO_CHAR(AL.ACCESS_DATE, 'MM/DD'), TRUNC(AL.ACCESS_DATE)

    ORDER BY TRUNC(AL.ACCESS_DATE)
</select>
```

3

```
<!-- 오래된 접속 로그 삭제 (7일 이전) -->
<delete id="deleteOldAccessLogs">
    DELETE FROM ACCESS_LOG
    WHERE ACCESS_DATE < TRUNC(SYSDATE) - 7
</delete>
```

2

AL.ACCESS_DATE >= TRUNC(SYSDATE) - 7 : 최근 7일 로그만 조회
NVL(M.ADMIN_YN, 'N') = 'N' : 관리자의 접속은 통계에서 제외
GROUP BY TO_CHAR(AL.ACCESS_DATE, 'MM/DD'),
TRUNC(AL.ACCESS_DATE)
→ 하루 단위로 로그인 수 집계하는 SELECT문

3

ACCESS_DATE < TRUNC(SYSDATE) - 7 : 최근 7일 이전 로그 모두 삭제
→ 이 쿼리문은 스케줄러를 통해 자정마다 삭제하도록 처리하는 DELETE문

* 코드리뷰 - 신선호

1. 프론트가 영수증 이미지 전송 → 컨트롤러가 받는 구간

```
@PostMapping("/receipt")
public ResponseEntity<?> extractReceiptText(@RequestParam("image") MultipartFile image) {
    try {
        1 if (image.isEmpty()) {
            Map<String, Object> errorResponse = new HashMap<>();
            errorResponse.put(key: "success", value: false);
            errorResponse.put(key: "error", value: "이미지 파일이 비어있습니다.");
            return ResponseEntity.badRequest().body(errorResponse);
        }
        String contentType = image.getContentType();
        2 if (contentType == null || !contentType.startsWith(prefix: "image/")) {
            Map<String, Object> errorResponse = new HashMap<>();
            errorResponse.put(key: "success", value: false);
            errorResponse.put(key: "error", value: "이미지 파일만 업로드 가능합니다.");
            return ResponseEntity.badRequest().body(errorResponse);
        }
        3 OcrResponse ocrResponse = ocrService.extractTextFromReceipt(image);
    }
}
```

1. 파일이 비었는지 검증
2. 이미지 mime 타입인지 검증
3. 서비스로 ocr 처리 위임

* 코드리뷰 - 신선호

2. NAVER OCR API 호출

```
1 OcrRequest ocrRequest = new OcrRequest(  
    version: "V2",  
    String.valueOf(System.currentTimeMillis()),  
    System.currentTimeMillis(),  
    Collections.singletonList(new OcrImageInfo(fileExtension, originalFilename))  
);  
  
String messageJson = objectMapper.writeValueAsString(ocrRequest);  
  
2 RequestBody requestBody = new MultipartBody.Builder()  
    .setType(MultipartBody.FORM)  
    .addFormDataPart(arg0: "message", messageJson)  
    .addFormDataPart(  
        arg0: "file",  
        originalFilename,  
        RequestBody.create(  
            imageFile.getBytes(),  
            MediaType.parse(imageFile.getContentType())  
        )  
    )  
    .build();
```

1. 네이버 OCR 요청 바디 생성

2. 멀티파트 요청(body)에 message + file 함께 담기

* 코드리뷰 - 신선호

2. NAVER OCR API 호출

```
3 Request request = new Request.Builder()
    .url(ocrApiUrl)
    .addHeader(arg0: "X-OCR-SECRET", secretKey)
    .post(requestBody)
    .build();

try (Response response = httpClient.newCall(request).execute()) {
    if (!response.isSuccessful()) {
        String errorBody = response.body() != null ? response.body().string() : "No error body";
        throw new IOException("OCR API 호출 실패: " + response.code() + " - " + errorBody);
    }

    String responseBody = response.body().string();
    4 return objectMapper.readValue(responseBody, valueType: OcrResponse.class);
}
```

3. 네이버 OCR API 호출

4. 응답 JSON → OcrResponse로 변환
후 컨트롤러로 반환

* 코드리뷰 - 신선희

3. 프론트가 OCR 텍스트로 병원 인증 요청 → 컨트롤러

```
// ===== OCR 텍스트로 병원 인증 =====  
@PostMapping("/verifyReceipt")  
public ResponseEntity<OcrVerifyResponse> verifyReceipt(  
    @RequestBody OcrVerifyRequest request) {  
    1 OcrVerifyResponse result = ocrService.verifyReceiptByText(request.getOcrText());  
    2 return ResponseEntity.ok(result);  
}
```

1. 프론트에서 넘어온 ocrText 꺼내기
2. 서비스 결과 그대로 반환

* 코드리뷰 - 신선호

4. OCR 텍스트에서 병원명 라인 찾기 + 키워드 추출

```
// ===== 2) OCR 텍스트만으로 병원 찾기 =====
public OcrVerifyResponse verifyReceiptByText(String ocrText) {
    // 1) 빈 텍스트 예외 처리
    1 if (ocrText == null || ocrText.trim().isEmpty()) {
        return new OcrVerifyResponse(success: false, verified: false,
            message: "추출된 텍스트가 비어 있습니다.", hospitalId: null, hospitalName: null);
    }
    2 // 2) 줄 단위로 분리해서 병원명 있을 법한 줄 찾기
    String[] lines = ocrText.split(regex: "\\r?\\n");
    String hospitalLine = null;

    for (String line : lines) {
        if (line == null) continue;
        String t = line.trim();
        if (t.isEmpty()) continue;

        // 병원 관련 키워드
        if (t.contains(s: "병원") || t.contains(s: "의원") ||
            t.contains(s: "치과") || t.contains(s: "한의원")) {
            hospitalLine = t;
            break;
        }
    }

    if (hospitalLine == null) {
        return new OcrVerifyResponse(success: false, verified: false,
            message: "영수증에서 병원명을 찾지 못했습니다.", hospitalId: null, hospitalName: null);
    }
}
```

1. 빈 텍스트 예외 처리

2. 줄 단위로 분리해서 병원명 있을

* 코드리뷰 - 신선호

4. OCR 텍스트에서 병원명 라인 찾기 + 키워드 추출

```
// 3) 특수문자 제거해서 병원명 키워드 추출
3 String keyword = hospitalLine.replaceAll(regex: "[^0-9a-zA-Z가-힣]", replacement: "");
log.info(format: "[OCR VERIFY] hospitalLine={}, keyword={}", hospitalLine, keyword);

// 4) DB에서 병원 후보 조회
4 java.util.List<HospitalOcrInfo> candidates =
    ocrMapper.searchHospitalByNameForOCR(keyword);

if (candidates == null || candidates.isEmpty()) {
    return new OcrVerifyResponse(success: false, verified: false,
        message: "해당 병원을 찾을 수 없습니다.", hospitalId: null, hospitalName: null);
}
```

3. 특수문자 제거해서 병원명 키워드 추출

4. DB에서 병원 후보 조회

* 코드리뷰 - 신선호

5. 후보 병원 점수 계산 및 최종 인증 여부 결정

```
// 4) 정규화 텍스트 준비
String normText = normalize(ocrText);

HospitalOcrInfo best = null;
int bestScore = -1;

for (HospitalOcrInfo info : candidates) {
    if (info == null) continue;

    1 String rawName = info.getHospitalName() != null ? info.getHospitalName() : "";
    1 String mainName = rawName.split(regex: "[\\(\\[\\]]")[0]; // "푸른치과의원(노원점)" -> "푸른치과의원"
    2 String normName = normalize(mainName);
    2 String normAddrFull = normalize(info.getAddress());

    3 boolean nameMatch = !normName.isEmpty() && normText.contains(normName);
    int addrScore = calcAddressMatchScore(info.getAddress(), normText);

    // 점수: 이름매칭되면 +10, 주소 토큰 매칭 수를 더함
    int score = (nameMatch ? 10 : 0) + addrScore;

    log.info(format: "[OCR VERIFY] candidate id={}, name={}, addr={}, nameMatch={}, addrScore={}, t
        info.getHospitalId(), rawName, info.getAddress(), nameMatch, addrScore, score);

    if (score > bestScore) {
        bestScore = score;
        best = info;
    }
}
```

1. 병원명에서 지점 정보 괄호 제거
2. 병원명 / 주소 정규화
3. 이름 일치 여부 + 주소 일치 점수 계산

* 코드리뷰 - 신선희

5. 후보 병원 점수 계산 및 최종 인증 여부 결정

```
// 5) 최종 판단
4 if (best == null || bestScore <= 0) {
    return new OcrVerifyResponse(
        success: true,
        verified: false,
        message: "영수증의 병원명/주소가 병원 정보와 충분히 일치하지 않습니다.",
        hospitalId: null,
        hospitalName: null
    );
}

return new OcrVerifyResponse(
    success: true,
    verified: true,
    message: "영수증 인증이 완료되었습니다.",
    best.getHospitalId(),
    best.getHospitalName()
);
}
```

4. 최종 판단

* 코드리뷰 - 신선호

6. MAPPER DB에서 병원 정보 조회

```
@Mapper
public interface OCRMapper {
    // ✅ OCR 텍스트에서 찾은 병원명으로 검색
    List<HospitalOcrInfo> searchHospitalByNameForOCR(@Param("nameKeyword") String nameKeyword);
    // 전체 병원 목록
    List<HospitalOcrInfo> findAllHospitalsForOCR();
}
```

searchHospitalByNameForOCR()

서비스에서 전달한 nameKeyword 로 병원 테이블 검색.

이름 LIKE 검색 + 필요하면 주소 조건까지 포함한 SQL을 Mapper XML에서 구현합니다.

그 결과를 HospitalOcrInfo 리스트로 반환하여

서비스에서 점수 계산 로직에 사용합니다.

findAllHospitalsForOCR() 는 필요 시 전체 병원 목록을 OCR 용으로 사용할 수 있는 보조 메소드.

코드 흐름

요청

→ Controller 메소드

→ Service 로직

→ Mapper 호출

→ 결과 응답

* 코드리뷰 - 임동균

소셜 로그인

1 OAuth2 Social Login Configuration

```
oauth.redirect-uri=  
# Kakao  
oauth.kakao.client-id=  
# Naver  
oauth.naver.client-id=  
oauth.naver.client-secret=
```

```
2 Value("${oauth.redirect-uri}")  
private String redirectUri;  
@Value("${oauth.kakao.client-id}")  
private String kakaoClientId;  
@Value("${oauth.naver.client-id}")  
private String naverClientId;  
@Value("${oauth.naver.client-secret}")  
private String naverClientSecret;
```

1. API key 보안을 위해 application.properties에 저장
2. @Value를 활용하여 값을 주입하고 private 필드에 저장해서 해당 클래스에서 사용하는 방식을 사용

* 코드리뷰 - 임동균

소셜 로그인

```
// 1. 액세스 토큰 받기
String tokenUrl = "https://kauth.kakao.com/oauth/token";

HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);

1 MultiValueMap<String, String> params = new LinkedMultiValueMap<>();
params.add("grant_type", "authorization_code");
params.add("client_id", kakaoClientId);
params.add("redirect_uri", redirectUri);
params.add("code", code);

HttpEntity<MultiValueMap<String, String>> request = new HttpEntity<>(params, headers);
2 ResponseEntity<Map> tokenResponse = restTemplate.postForEntity(tokenUrl, request, Map.class);

if (tokenResponse.getBody() == null) {
    throw new RuntimeException("Failed to get token response from Kakao");
}

String accessToken = (String) tokenResponse.getBody().get("access_token");

if (accessToken == null) {
    throw new RuntimeException("Failed to get access token from Kakao: " + tokenResponse);
}
```

1. 사용자 정보와 API KEY값을 파라미터에 담음
2. 토큰 발급 URL로 발급 요청 전송

* 코드리뷰 - 임동균

소셜 로그인

```
// 2. 사용자 정보 가져오기
String userInfoUrl = "https://kapi.kakao.com/v2/user/me";

HttpHeaders userHeaders = new HttpHeaders();
1 userHeaders.setBearerAuth(accessToken);

HttpEntity<String> userRequest = new HttpEntity<>(userHeaders);
ResponseEntity<Map> userResponse =
2 restTemplate.exchange(userInfoUrl, HttpMethod.GET, userRequest, Map.class);

if (userResponse.getBody() == null || userResponse.getBody().get("id") == null) {
    throw new RuntimeException("Failed to get user info from Kakao");
}

return String.valueOf(userResponse.getBody().get("id"));
```

1. 발급받은 토큰값을 필드에 저장

2. 사용자가 동의한 정보와 토큰을 같이 사용자 정보 URL에 전송

단, 사업자가 아니면 회원정보를 제공해주지않아서
사용불가능

* 코드리뷰 - 임동균

Cloudflare 이미지 저장

```
1 Cloudflare R2 Configuration
cloud.r2.endpoint=https
cloud.r2.access-key=7de
cloud.r2.secret-k
cloud.r2.bucket
cloud.r2.public-url=htt

2 @Value("${cloud.r2.endpoint}")
private String endpoint;
@Value("${cloud.r2.access-key}")
private String accessKey;
@Value("${cloud.r2.secret-key}")
private String secretKey;
@Value("${cloud.r2.bucket}")
private String bucketName;
@Value("${cloud.r2.public-url}")
private String publicUrl;
```

1. API key 보안을 위해 application.properties에 저장
2. @Value를 활용하여 값을 주입하고 private 필드에 저장해서 해당 클래스에서 사용하는 방식을 사용

* 코드리뷰 - 임동균

Cloudflare 이미지 저장

```
// 이미지가 있으면 Cloudflare R2에 업로드
if (image != null && !image.isEmpty()) {
    1 String timestamp = new SimpleDateFormat("yyyyMMddHHmmss")
      .format(new Date());
    String extension = "";
    String originalFilename = image.getOriginalFilename();
    if (originalFilename != null && originalFilename
        .contains(".")) {
        extension = originalFilename
            .substring(originalFilename.lastIndexOf("."));
    }
    String fileName = timestamp + extension;
    2 String imageUrl = r2Service
      .uploadFile(image, "__healbot__/disease/img", fileName);
    disease.setImageUrl(imageUrl);
}
```

1. 파일명을 timestamp로 하여 중복없이 생성
2. 이미지 정보와, 폴더명, 파일명을 Service로 전송

* 코드리뷰 - 임동균

Cloudflare 이미지 저장

```
public String uploadFile(MultipartFile file, String folder) throws IOException {
    if (file == null || file.isEmpty()) {
        throw new IllegalArgumentException("파일이 비어있습니다.");
    }

    String originalFilename = file.getOriginalFilename();
    String extension = originalFilename != null && originalFilename.contains(".")
        ? originalFilename.substring(originalFilename.lastIndexOf("."))
        : "";

    1 String fileName = UUID.randomUUID().toString() + extension;
    2 String key = folder.endsWith("/") ? folder + fileName : folder + "/" + fileName;
    3 try (InputStream inputStream = file.getInputStream()) {
        PutObjectRequest putObjectRequest = PutObjectRequest.builder()
            .bucket(bucketName)
            .key(key)
            .contentType(file.getContentType())
            .contentLength(file.getSize())
            .build();

        s3Client.putObject(putObjectRequest, RequestBody.fromInputStream(inputStream));

        String fileUrl = publicUrl + "/" + key;
        return fileUrl;
    }
}
```

1. 파일명에 UUID를 활용하여 혹시나 모르는 중복제거
2. application.properties에 저장된 경로에 폴더명을 추가하여 경로지정
3. 수정된 파일명, 경로, 이미지를 API URL로 key값과 같이 전송



* 코드리뷰 - 남윤우

```
function scrollToSectionSmooth(index) {  
  1 if (index < 0 || index >= allScrollSections.length) return;  
  2 isScrolling = true;  
  currentSection = index;  
  
  allScrollSections[index].scrollIntoView({  
    3 behavior: "smooth",  
    block: "start",  
  });  
  
  4 updateScrollToTopButton(index);  
  
  setTimeout(() => {  
    5 isScrolling = false;  
  }, 1000);  
}
```

섹션 단위 스크롤

1. 인덱스 유효성 검사

- 섹션 범위를 벗어나는 요청은 무시
- 배열 범위 검사로 예외 방지
- 첫 섹션보다 위로 가거나 마지막 섹션보다 아래로 가는 것 방지

2. 스크롤 중복 방지 플래그

- isScrolling 플래그의 역할: 스크롤 애니메이션이 진행 중이면 다른 휠 입력을 무시해서 '한 섹션씩' 이동하도록 보장
- currentSection: 현재 위치한 섹션 인덱스 업데이트

3. 부드러운 스크롤 실행

- scrollIntoView(): 브라우저 네이티브 API 사용
- behavior: "smooth": CSS scroll-behavior와 동일한 부드러운 전환
- block: "start": 해당 섹션이 뷰포트의 맨 위에 오도록 스크롤

4. 맨 위로 버튼 업데이트

- 첫 번째 섹션(index === 0)이면 버튼 숨김
- 그 외 섹션에서는 버튼 표시

5. 스크롤 잠금 해제

- 1초(스크롤 애니메이션 시간) 후 플래그 해제
- 이 시간 동안 다른 스크롤 입력 무시

* 코드리뷰 - 남윤우

휠로 섹션 이동

```
1 const handleWheel = (e) => {  
  // 모달이 열려있으면 스크롤 막기  
  if (document.body.classList.contains('modal-open')) {  
    e.preventDefault();  
    return;  
  }  
  
2  e.preventDefault();  
3  if (isScrolling) return;  
  
4  if (e.deltaY > 0 && currentSection < allScrollSections.length - 1) {  
    scrollToSectionSmooth(currentSection + 1);  
  } else if (e.deltaY < 0 && currentSection > 0) {  
    scrollToSectionSmooth(currentSection - 1);  
  }  
};
```

1. 모달 감지

- 모달이 열려있으면 풀페이지 스크롤 비활성화
- 사용자가 모달 내용을 자유롭게 스크롤 가능

2. 기본 스크롤 동작 차단

- 브라우저의 일반적인 스크롤 막기
- 우리가 정의한 섹션 단위 스크롤만 동작

3. 스크롤 중복 방지

- 스크롤 애니메이션이 진행 중이면 새로운 입력 무시
- 빠르게 여러 번 휠을 돌려도 한 섹션씩만 이동

4. 휠 방향 감지

- $e.deltaY > 0$: 휠을 아래로 (다음 섹션으로)
- $e.deltaY < 0$: 휠을 위로 (이전 섹션으로)
- 경계 체크: 첫/마지막 섹션에서는 더 이상 이동 안 함

* 코드리뷰 - 남윤우

스크롤 시 헤더 색상 및 섹션 위치 감지

```
const handleScroll = () => {  
  1 if (!mainSection || !searchSection || !header) return;  
  
  2 const searchSectionEnd =  
    searchSection.offsetTop + searchSection.offsetHeight;  
  
  3 if (window.scrollY >= searchSectionEnd - 100) {  
    header.classList.add("scrolled");  
  } else {  
    header.classList.remove("scrolled");  
  }  
}
```

기능 1 : 헤더 배경색 변경

가독성과 UX 개선

1. DOM 요소 확인

- 필요한 요소들이 로드되지 않았으면 실행 안 함

2. 검색 섹션 끝 지점 계산

- offsetTop: 섹션의 시작 위치 (페이지 상단부터의 거리)

- offsetHeight: 섹션의 높이

- 합: 검색 섹션이 끝나는 Y 좌표

3. 헤더 스타일 토글

- window.scrollY: 현재 스크롤 위치

- 검색 섹션 끝에서 100px 전부터 헤더에 scrolled 클래스 추가

- CSS에서 .scrolled 클래스로 배경색, 텍스트 색상 등 변경

CSS 연동 (Header.css):

```
header.scrolled {  
  background: white;  
  color: #333;  
}
```

* 코드리뷰 - 남윤우

스크롤 시 헤더 색상 및 섹션 위치 감지

```
1 if (isScrolling) return;

2 let current = 0;
  const scrollPosition = window.pageYOffset; // 현재 스크롤 위치
  const windowHeight = window.innerHeight; // 브라우저 창 높이
  const documentHeight = document.documentElement.scrollHeight;
                                     // 전체 문서 높이

3 allScrollSections.forEach((section, index) => {
  if (!section) return;
  const sectionTop = section.offsetTop; // 섹션 시작 위치
  const sectionHeight = section.clientHeight; // 섹션 높이
  const checkPosition = scrollPosition + windowHeight / 2;
                                     // 화면 중앙 유지

  if (
    checkPosition >= sectionTop &&
    checkPosition < sectionTop + sectionHeight
  ) {
    current = index;
  }
});

4 if (scrollPosition + windowHeight >= documentHeight - 50) {
  current = allScrollSections.length - 1;
}

5 updateScrollToTopButton(current);
  currentSection = current;
};
```

기능 2: 현재 섹션 위치 추적

1. 스크롤 중 위치 추적 스킵

- 섹션 단위 스크롤 애니메이션 중에는 위치 추적 안 함
- 애니메이션이 끝난 후에만 정확한 위치 계산

2. 필요한 값 계산

3. 각 섹션 순회하며 현재 위치 판단

- 현재 화면 중앙이 어느 섹션에 있는지 판단

4. 페이지 하단 감지

- 페이지 맨 아래에 도달하면 마지막 섹션으로 인식
- -50: 오차 범위 (정확히 끝까지 안 가도 인식)

5. 상태 업데이트

updateScrollToTopButton(current); // 버튼 표시/숨김
currentSection = current; // 현재 섹션 인덱스 저장



시연

* 소감



김재훈

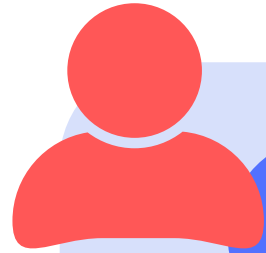
이번 프로젝트에서 접속 로그 시스템과 이메일 인증 기능을 구현하면서 실무 서비스의 핵심 요소들을 경험할 수 있었습니다.
특히 Spring Scheduler를 활용한 접속 로그 자동 처리와 Gmail SMTP를 통한 이메일 인증 코드 발송 구현 과정에서 비동기 처리와 보안을 고려한 개발 역량을 키울 수 있었던 의미 있는 프로젝트였습니다.



남윤우

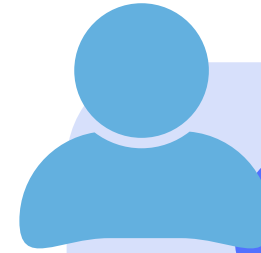
카카오맵 API를 연동하여 병원 마커 표시, 필터링, 바운드 기반 검색 등 실시간 지도 서비스를 구현하며 외부 API 활용 능력을 향상시킬 수 있었습니다. 또한 메인페이지에 섹션별 스크롤 네비게이션과 자동 스크롤 기능을 개발하면서 React Hooks를 활용한 상태 관리와 사용자 친화적인 인터페이스 구현 경험을 쌓았습니다.

* 소감



신선호

HealBot에서 마이페이지, 리뷰(OCR 인증), 커뮤니티 기능을 맡아 개발 흐름을 폭넓게 경험할 수 있었습니다. 마이페이지는 다양한 정보 관리로 데이터 구조를 이해하는 데 도움이 되었고, 리뷰 페이지는 OCR 예외 처리 덕분에 서버 로직을 깊게 다질 수 있었습니다. 커뮤니티 구현을 통해 실제 서비스 같은 게시판 기능을 직접 만들어보며 실전 감각을 키울 수 있었습니다.



임동균

이번 프로젝트에서 카카오와 네이버 소셜 로그인 기능을 구현하면서 실제 서비스 수준의 인증 시스템을 경험할 수 있었습니다. 특히 Azure 클라우드 환경에 실제 배포하고 도메인을 연결하는 과정에서 OPEN-API들의 리다이렉트 URI 변경과 같은 이슈들을 해결하면서 실질적인 경험을 쌓을 수 있었던 프로젝트 였습니다.

QnA

THANK YOU



감사합니다.